

Angular

Complete Learning Guide

From Scratch - Associate Software Engineer Level

Java Full Stack - L1 and L2 Interview Preparation

Topics: TypeScript - Components - Directives - Services - Routing
Forms - HTTP - RxJS - Spring Boot Integration - Interview Q&A

A structured roadmap for full stack developers

Table of Contents

1. Introduction to Angular
2. Prerequisites: HTML, CSS, JavaScript
3. TypeScript Fundamentals
4. Setting Up Angular (Node, CLI, Project)
5. Angular Architecture
6. Components and Templates
7. Data Binding
8. Directives (Structural and Attribute)
9. Pipes
10. Services and Dependency Injection
11. Component Lifecycle Hooks
12. Component Communication
13. Modules and Lazy Loading
14. Routing and Navigation
15. Forms - Template-Driven and Reactive
16. HTTP Client and REST API Integration
17. Observables and RxJS
18. Interceptors and Error Handling
19. Authentication and JWT (with Spring Boot)
20. Angular + Spring Boot Full Stack Integration
21. State Management Basics
22. Unit Testing Essentials
23. Build, Optimization and Deployment
24. Best Practices and Style Guide
25. L1 Interview Questions (Beginner)
26. L2 Interview Questions (Intermediate)
27. Coding & Scenario-Based Questions
28. Final Preparation Checklist

1. Introduction to Angular

Angular is a TypeScript-based open-source front-end web application framework developed and maintained by Google. It is used to build single-page applications (SPAs) and dynamic web apps with a modular, component-based architecture. Angular provides everything you need to build large-scale enterprise applications: components, routing, forms, HTTP communication, dependency injection, testing utilities, and a CLI.

Why Learn Angular?

- Backed by Google with long-term support and a stable release cycle.
- Used widely in enterprise applications (banking, healthcare, e-commerce).
- Strong TypeScript support brings type safety and tooling.
- Opinionated framework: clear conventions reduce decision fatigue.
- Excellent integration with backend frameworks like Spring Boot, .NET, Node.js.
- Powerful CLI for scaffolding, building, testing, and deploying.

AngularJS vs Angular

AngularJS (v1.x) is a legacy JavaScript framework. Angular (v2 and beyond) is a complete rewrite using TypeScript with a component-based architecture. When people say 'Angular' today, they usually mean Angular 2+ (currently Angular 17+).

Aspect	AngularJS (1.x)	Angular (2+)
Language	JavaScript	TypeScript
Architecture	MVC	Component-based
Mobile Support	Limited	Yes
DI System	Single injector	Hierarchical injectors
Performance	Slower (2-way binding)	Faster (change detection, AOT)
CLI Tool	No official CLI	Powerful Angular CLI

2. Prerequisites: HTML, CSS, JavaScript

Before diving into Angular, you should be comfortable with the building blocks of the web. You don't need to be an expert, but the following are essential.

HTML Essentials

- Semantic tags: header, main, section, article, footer, nav.
- Forms and form elements: input, select, textarea, button.
- Attributes vs properties (Angular treats them differently).
- Block vs inline elements; the DOM tree.

CSS Essentials

- Selectors, specificity, inheritance, the cascade.
- Box model: margin, border, padding, content.
- Flexbox and Grid for layout.
- Responsive design with media queries.
- CSS variables and preprocessors (SCSS) - commonly used with Angular.

JavaScript (ES6+) Essentials

- let, const, var and block scoping.
- Arrow functions and lexical 'this'.
- Template literals: `Hello \${name}`.
- Destructuring: `const {a, b} = obj; const [x, y] = arr;`
- Spread and rest operators: `...args`.
- Promises and async/await.
- ES6 Modules: `import / export`.
- Classes, inheritance, and constructors.
- Array methods: `map, filter, reduce, find, forEach`.

3. TypeScript Fundamentals

Angular is built with TypeScript. TypeScript is a strongly-typed superset of JavaScript that compiles to plain JavaScript. It catches errors at compile time and provides powerful tooling like autocomplete and refactoring.

Basic Types

```
let age: number = 25;
let name: string = 'Ravi';
let isActive: boolean = true;
let scores: number[] = [90, 85, 78];
let tags: Array<string> = ['java', 'angular'];
let anything: any = 'can be anything';
let nothing: void = undefined;
```

Interfaces and Types

```
interface Employee {
  id: number;
  name: string;
  email?: string;    // optional
  readonly role: string; // immutable
}

const emp: Employee = {
  id: 1, name: 'Asha', role: 'Developer'
};
```

Classes and Access Modifiers

```
class Person {
  private id: number;
  public name: string;
  protected age: number;

  constructor(id: number, name: string, age: number) {
    this.id = id;
    this.name = name;
    this.age = age;
  }

  greet(): string {
    return `Hello, ${this.name}`;
  }
}
```

Generics

```
function identity<T>(value: T): T {
  return value;
}

const num = identity<number>(42);
const str = identity<string>('hello');

interface ApiResponse<T> {
  status: number;
  data: T;
}
```

```
}
```

Enums, Union and Intersection Types

```
enum Role { Admin, User, Guest }  
let r: Role = Role.Admin;  
  
type Status = 'active' | 'inactive' | 'pending';  
let s: Status = 'active';  
  
type A = { name: string };  
type B = { age: number };  
type AB = A & B; // must have both
```

Decorators (used heavily in Angular)

Decorators are special declarations that can be attached to classes, methods, or properties. Angular uses them to mark classes as Components, Services, Modules, etc.

```
@Component({  
  selector: 'app-hello',  
  template: '<h1>Hello</h1>'  
})  
export class HelloComponent {}
```

4. Setting Up Angular

Step 1: Install Node.js (LTS version) from nodejs.org. Verify:

```
node --version
npm --version
```

Step 2: Install Angular CLI globally.

```
npm install -g @angular/cli
ng version
```

Step 3: Create a new Angular project.

```
ng new my-app
# Answer prompts:
#   Routing? yes
#   Stylesheet? CSS or SCSS

cd my-app
ng serve --open
```

Step 4: Project Structure

```
my-app/
  src/
    app/
      app.component.ts      <- root component
      app.component.html
      app.component.css
      app.module.ts         <- root module
      app-routing.module.ts <- routes
    assets/                 <- static files
    environments/           <- env configs
    index.html              <- single HTML page
    main.ts                 <- bootstrap entry
    styles.css              <- global styles
  angular.json              <- workspace config
  package.json              <- dependencies
  tsconfig.json             <- TypeScript config
```

Common CLI Commands

Command	Purpose
ng serve	Run dev server (default http://localhost:4200)
ng build	Build the project
ng build --configuration production	Production build (minified, AOT)
ng generate component <name>	Create a component
ng g s <name>	Create a service (shorthand)
ng g m <name>	Create a module
ng g d <name>	Create a directive
ng g p <name>	Create a pipe
ng g guard <name>	Create a route guard
ng test	Run unit tests (Karma + Jasmine)

ng e2e	Run end-to-end tests
ng lint	Lint the codebase

5. Angular Architecture

An Angular application is composed of building blocks that work together. Understanding how these pieces fit together is essential for interviews.

Core Building Blocks

- bullet **Modules (NgModule):** Containers that group related components, directives, pipes, and services. Every app has at least one root module - `AppModule`.
- bullet **Components:** The basic UI building block. Each component has a TypeScript class, an HTML template, and styles.
- bullet **Templates:** HTML with Angular syntax (interpolation, bindings, directives) that defines how a component renders.
- bullet **Directives:** Add behavior to DOM elements. Three kinds: components, structural (`*ngIf`), attribute (`ngClass`).
- bullet **Services:** Reusable logic (data fetching, business rules) shared across components, typically injected via Dependency Injection.
- bullet **Dependency Injection (DI):** Angular's way of providing instances of services to classes that need them.
- bullet **Routing:** Maps URLs to components for single-page navigation.
- bullet **Metadata:** Decorators like `@Component`, `@NgModule`, `@Injectable` describe how to process a class.

How an Angular App Starts

- bullet Browser loads `index.html` (the single page).
- bullet `main.ts` bootstraps the `AppModule`.
- bullet `AppModule` loads the `AppComponent`.
- bullet `AppComponent`'s `selector()` is replaced with its template.
- bullet Routes are resolved and child components render inside .

```
// main.ts
import { platformBrowserDynamic } from '@angular/platform-browser-dynamic';
import { AppModule } from './app/app.module';

platformBrowserDynamic().bootstrapModule(AppModule)
  .catch(err => console.error(err));
```

6. Components and Templates

Components are the heart of Angular. A component controls a portion of the screen (a view) through its template, class, and styles.

Creating a Component

```
ng generate component employee
# shorthand: ng g c employee
```

Anatomy of a Component

```
// employee.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-employee',
  templateUrl: './employee.component.html',
  styleUrls: ['./employee.component.css']
})
export class EmployeeComponent {
  title = 'Employee List';
  employees = [
    { id: 1, name: 'Ravi' },
    { id: 2, name: 'Sita' }
  ];
}
```

Template Example

```
<!-- employee.component.html -->
<h2>{{ title }}</h2>
<ul>
  <li *ngFor="let emp of employees">
    {{ emp.id }} - {{ emp.name }}
  </li>
</ul>
```

Component Metadata Properties

- selector:** CSS selector used in HTML to insert the component (e.g.).
- templateUrl / template:** External HTML file or inline string.
- styleUrls / styles:** Component-scoped styles.
- providers:** Services available only to this component and its children.
- encapsulation:** ViewEncapsulation strategy (Emulated, None, ShadowDom).
- changeDetection:** Default or OnPush for performance.

7. Data Binding

Data binding is the synchronization between the component class and the view. Angular supports four primary forms of data binding.

Type	Syntax	Direction
Interpolation	{{ value }}	Component to View
Property Binding	[property]="value"	Component to View
Event Binding	(event)="handler()"	View to Component
Two-Way Binding	[(ngModel)]="value"	Both Ways

1. Interpolation

```
// component.ts
name = 'Angular';

<!-- template -->
<h1>Hello {{ name }}</h1>
<p>Total: {{ 2 + 3 }}</p>
```

2. Property Binding

```
<img [src]="imgUrl" [alt]="imgAlt">
<button [disabled]="isDisabled">Save</button>
<div [class.active]="isActive">Tab</div>
<div [style.color]="textColor">Text</div>
```

3. Event Binding

```
<button (click)="onSave()">Save</button>
<input (input)="onChange($event)">
<form (submit)="onSubmit()">...</form>
```

4. Two-Way Binding (Banana in a Box)

Requires FormsModule import in your NgModule.

```
<input [(ngModel)]="username">
<p>Hello, {{ username }}</p>

// Equivalent to:
<input [ngModel]="username" (ngModelChange)="username = $event">
```

8. Directives

Directives are classes that add additional behavior to elements in your templates. Angular has three kinds of directives.

Types of Directives

Component: A directive with a template (the most common kind).

Structural directives: Change the DOM layout by adding/removing elements (prefixed with *).

Attribute directives: Change the appearance or behavior of an existing element.

Common Structural Directives

```
<!-- *ngIf -->
<p *ngIf="isLoggedIn">Welcome back!</p>
<p *ngIf="isLoggedIn; else loggedOut">Welcome</p>
<ng-template #loggedOut><p>Please login</p></ng-template>

<!-- *ngFor -->
<li *ngFor="let item of items; let i = index; trackBy: trackById">
  {{ i + 1 }}. {{ item.name }}
</li>

<!-- *ngSwitch -->
<div [ngSwitch]="status">
  <p *ngSwitchCase="'active'">Active</p>
  <p *ngSwitchCase="'inactive'">Inactive</p>
  <p *ngSwitchDefault>Unknown</p>
</div>
```

Common Attribute Directives

```
<!-- ngClass -->
<div [ngClass]="{'active': isActive, 'error': hasError}">...</div>

<!-- ngStyle -->
<div [ngStyle]="{'color': textColor, 'font-size.px': size}">...</div>

<!-- ngModel (with FormsModule) -->
<input [(ngModel)]="name">
```

Custom Attribute Directive

```
// highlight.directive.ts
import { Directive, ElementRef, HostListener, Input } from '@angular/core';

@Directive({ selector: '[appHighlight]' })
export class HighlightDirective {
  @Input() appHighlight = 'yellow';

  constructor(private el: ElementRef) {}

  @HostListener('mouseenter') onEnter() {
    this.el.nativeElement.style.backgroundColor = this.appHighlight;
  }
  @HostListener('mouseleave') onLeave() {
    this.el.nativeElement.style.backgroundColor = null;
  }
}
```

```
    }  
  }
```

```
<!-- usage -->  
<p appHighlight="lightblue">Hover me</p>
```

Why Only One Structural Directive Per Element?

Angular forbids two structural directives on the same element because the order in which they apply would be ambiguous. Wrap one in instead:

```
<ng-container *ngIf="items">  
  <li *ngFor="let item of items">{{ item }}</li>  
</ng-container>
```

9. Pipes

Pipes transform displayed values in templates without changing the underlying data. Use the pipe symbol (|).

Built-in Pipes

```
{{ name | uppercase }}           &lt;!-- RAVI --&gt;
{{ name | lowercase }}           &lt;!-- ravi --&gt;
{{ name | titlecase }}           &lt;!-- Ravi Kumar --&gt;
{{ today | date:'dd/MM/yyyy' }}  &lt;!-- 13/06/2026 --&gt;
{{ price | currency:'INR':'symbol' }}
{{ 0.756 | percent:'1.0-2' }}    &lt;!-- 75.6% --&gt;
{{ 12345.678 | number:'1.2-2' }} &lt;!-- 12,345.68 --&gt;
{{ obj | json }}
{{ text | slice:0:10 }}
{{ data$ | async }}              &lt;!-- subscribes & cleans up --&gt;
```

Chaining Pipes

```
{{ today | date:'fullDate' | uppercase }}
```

Custom Pipe

```
// reverse.pipe.ts
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({ name: 'reverse' })
export class ReversePipe implements PipeTransform {
  transform(value: string): string {
    return value ? value.split('').reverse().join('') : '';
  }
}

&lt;!-- usage --&gt;
&lt;p&gt;{{ 'hello' | reverse }}&lt;/p&gt;    &lt;!-- olleh --&gt;
```

Pure vs Impure Pipes

Pure (default): Re-evaluated only when input reference changes. Fast.

Impure: Re-evaluated on every change detection cycle. Use sparingly. Set pure: false in the @Pipe decorator.

10. Services and Dependency Injection

Components should focus on presentation. Business logic, data fetching, and shared functionality belong in services. Angular's Dependency Injection (DI) system provides instances of these services wherever they are needed.

Creating a Service

```
ng generate service services/employee
# creates employee.service.ts
```

Service Example

```
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' }) // app-wide singleton
export class EmployeeService {
  private employees = [
    { id: 1, name: 'Ravi' },
    { id: 2, name: 'Sita' }
  ];

  getAll() { return this.employees; }
  getById(id: number) {
    return this.employees.find(e => e.id === id);
  }
  add(emp: any) { this.employees.push(emp); }
}
```

Injecting a Service into a Component

```
@Component({...})
export class EmployeeListComponent implements OnInit {
  employees: any[] = [];

  constructor(private empService: EmployeeService) {}

  ngOnInit(): void {
    this.employees = this.empService.getAll();
  }
}
```

Provider Scopes

- **providedIn: 'root'** - singleton across the whole app (recommended).
- **providedIn: SomeModule** - one instance per loaded module.
- **providers in @Component** - new instance per component (and its subtree).
- **providedIn: 'any'** - separate instance per lazy-loaded module.

Hierarchical Injectors

Angular maintains a tree of injectors that mirrors the component tree. When a class requests a dependency, Angular looks up the tree starting from the local injector. This is how you can override a service for a specific subtree.

11. Component Lifecycle Hooks

Each component has a lifecycle managed by Angular. You can tap into key moments by implementing lifecycle interfaces.

Hook	When it runs	Common use
ngOnChanges	When @Input properties change	React to input changes
ngOnInit	Once, after first ngOnChanges	Init data, HTTP calls
ngDoCheck	Every change detection cycle	Custom change detection
ngAfterContentInit	After projected content init	Access @ContentChild
ngAfterContentChecked	After projected content check	Rarely used
ngAfterViewInit	After view init	Access @ViewChild / DOM
ngAfterViewChecked	After every view check	Rarely used
ngOnDestroy	Just before component destruction	Unsubscribe, cleanup

Example Usage

```
import { Component, OnInit, OnDestroy } from '@angular/core';
import { Subscription } from 'rxjs';

@Component({...})
export class TimerComponent implements OnInit, OnDestroy {
  sub!: Subscription;

  ngOnInit(): void {
    this.sub = interval(1000).subscribe(n => console.log(n));
  }

  ngOnDestroy(): void {
    this.sub.unsubscribe(); // important - prevents memory leak
  }
}
```


12. Component Communication

Parent to Child - @Input

```
// child.component.ts
@Component({ selector: 'app-child', template: '<p>{{ name }}</p>' })
export class ChildComponent {
  @Input() name!: string;
}

<!-- parent.component.html -->
<app-child [name]="parentName"></app-child>
```

Child to Parent - @Output and EventEmitter

```
// child.component.ts
@Component({ selector: 'app-child', template: '<button (click)="send()">Send</button>' })
export class ChildComponent {
  @Output() messageEvent = new EventEmitter<string>();
  send() { this.messageEvent.emit('Hello from child'); }
}

<!-- parent.component.html -->
<app-child (messageEvent)="receive($event)"></app-child>
```

Parent accesses Child - @ViewChild

```
@Component({...})
export class ParentComponent implements AfterViewInit {
  @ViewChild(ChildComponent) child!: ChildComponent;

  ngAfterViewInit(): void {
    console.log(this.child.someProperty);
  }
}
```

Sibling / Unrelated Components - via Service with Subject

```
// shared.service.ts
@Injectable({ providedIn: 'root' })
export class SharedService {
  private messageSubject = new Subject<string>();
  message$ = this.messageSubject.asObservable();
  send(msg: string) { this.messageSubject.next(msg); }
}

// component-a (sender)
this.shared.send('hi');

// component-b (receiver)
this.shared.message$.subscribe(m => this.text = m);
```

13. Modules and Lazy Loading

An NgModule groups related components, directives, pipes, and services. Every Angular application has at least one module - the root AppModule.

Structure of NgModule

```
@NgModule({
  declarations: [ AppComponent, EmployeeComponent ],
  imports:      [ BrowserModule, FormsModule, HttpClientModule, AppRoutingModule ],
  providers:    [ EmployeeService ],
  bootstrap:    [ AppComponent ]
})
export class AppModule {}
```

declarations: Components, directives, pipes belonging to this module.

imports: Other modules whose exports we use here.

providers: Service providers available within this module.

exports: What this module exposes to importers.

bootstrap: Root component(s) - only in AppModule.

Feature Modules

Group related functionality (e.g. UserModule, OrderModule) into separate feature modules. This improves organization and enables lazy loading.

```
ng generate module orders --route orders --module app.module
# Creates orders module with lazy-loaded route configuration
```

Lazy Loading

```
// app-routing.module.ts
const routes: Routes = [
  { path: '', component: HomeComponent },
  {
    path: 'orders',
    loadChildren: () =>
      import('./orders/orders.module').then(m => m.OrdersModule)
  }
];
```

Lazy loading splits the JavaScript bundle so users only download the code for routes they visit. This dramatically improves initial load time.

Shared Modules

A SharedModule typically exports common pipes, directives, and small components used across feature modules. Import it where needed.

14. Routing and Navigation

The Angular Router enables navigation between views within a single-page application without full page reloads.

Basic Setup

```
// app-routing.module.ts
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';

const routes: Routes = [
  { path: '', redirectTo: '/home', pathMatch: 'full' },
  { path: 'home', component: HomeComponent },
  { path: 'employees', component: EmployeeListComponent },
  { path: 'employees/:id', component: EmployeeDetailComponent },
  { path: '**', component: NotFoundComponent } // wildcard
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule {}
```

Router Outlet and Navigation Links

```
<nav>
  <a routerLink="/home" routerLinkActive="active">Home</a>
  <a routerLink="/employees" routerLinkActive="active">Employees</a>
</nav>
<router-outlet></router-outlet>
```

Programmatic Navigation

```
constructor(private router: Router) {}

goToDetail(id: number) {
  this.router.navigate(['/employees', id]);
}

// with query params
this.router.navigate(['/search'], { queryParams: { q: 'angular' } });
```

Reading Route Parameters

```
constructor(private route: ActivatedRoute) {}

ngOnInit(): void {
  // snapshot - one time
  const id = +this.route.snapshot.paramMap.get('id')!;

  // observable - if same route can be revisited with new params
  this.route.paramMap.subscribe(params => {
    this.id = +params.get('id')!;
  });
}
```

Route Guards

Guards control whether the user can enter, leave, or load a route. Common types: CanActivate, CanDeactivate, CanLoad, Resolve.

```
@Injectable({ providedIn: 'root' })
export class AuthGuard implements CanActivate {
  constructor(private auth: AuthService, private router: Router) {}

  canActivate(): boolean {
    if (this.auth.isLoggedIn()) return true;
    this.router.navigate(['/login']);
    return false;
  }
}

// in routes
{ path: 'admin', component: AdminComponent, canActivate: [AuthGuard] }
```

Child Routes

```
{
  path: 'dashboard',
  component: DashboardComponent,
  children: [
    { path: 'profile', component: ProfileComponent },
    { path: 'settings', component: SettingsComponent }
  ]
}
```

15. Forms - Template-Driven and Reactive

Angular provides two approaches for building forms: Template-Driven (simpler, declarative in template) and Reactive (more powerful, model-driven in TypeScript).

Template-Driven Forms

Import FormsModule in your NgModule.

```
<form #f="ngForm" (ngSubmit)="onSubmit(f)">
  <input name="username" [(ngModel)]="user.name" required minlength="3" #u="ngModel">
  <div *ngIf="u.invalid && u.touched">
    <span *ngIf="u.errors?.['required']">Name is required</span>
    <span *ngIf="u.errors?.['minlength']">Min 3 characters</span>
  </div>
  <input name="email" [(ngModel)]="user.email" required email>
  <button [disabled]="f.invalid">Submit</button>
</form>
```

Reactive Forms

Import ReactiveFormsModule in your NgModule.

```
// component.ts
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

export class SignupComponent implements OnInit {
  form!: FormGroup;

  constructor(private fb: FormBuilder) {}

  ngOnInit(): void {
    this.form = this.fb.group({
      name: ['', [Validators.required, Validators.minLength(3)]],
      email: ['', [Validators.required, Validators.email]],
      password: ['', [Validators.required, Validators.minLength(6)]]
    });
  }

  onSubmit(): void {
    if (this.form.valid) console.log(this.form.value);
  }
}

<!-- template -->
<form [formGroup]="form" (ngSubmit)="onSubmit()">
  <input formControlName="name">
  <div *ngIf="form.get('name')?.touched && form.get('name')?.invalid">
    Name is required (min 3 chars)
  </div>
  <input formControlName="email">
  <input type="password" formControlName="password">
  <button [disabled]="form.invalid">Sign up</button>
</form>
```

Template-Driven vs Reactive

Aspect	Template-Driven	Reactive
--------	-----------------	----------

Setup	FormsModule	ReactiveFormsModule
Form model	Implicit (in template)	Explicit (in class)
Data flow	Asynchronous	Synchronous
Best for	Simple forms	Complex, dynamic, testable forms
Validation	Directives in template	Validators in class
Testing	Harder	Easier

Custom Validators

```
// must contain at least one number
export function hasNumber(control: AbstractControl): ValidationErrors | null {
  return /\d/.test(control.value) ? null : { hasNumber: true };
}

// usage
password: ['', [Validators.required, hasNumber]]
```

16. HTTP Client and REST API Integration

Angular's `HttpClient` (from `@angular/common/http`) is the standard way to communicate with REST APIs - perfect for talking to a Spring Boot backend.

Setup

```
// app.module.ts
import { HttpClientModule } from '@angular/common/http';

@NgModule({ imports: [HttpClientModule, ...] })
export class AppModule {}
```

CRUD Service Example

```
// employee.service.ts
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

export interface Employee {
  id?: number;
  name: string;
  email: string;
  salary: number;
}

@Injectable({ providedIn: 'root' })
export class EmployeeService {
  private apiUrl = 'http://localhost:8080/api/employees';

  constructor(private http: HttpClient) {}

  getAll(): Observable<Employee[]> {
    return this.http.get<Employee[]>(this.apiUrl);
  }

  getById(id: number): Observable<Employee> {
    return this.http.get<Employee>(`${this.apiUrl}/${id}`);
  }

  create(emp: Employee): Observable<Employee> {
    return this.http.post<Employee>(this.apiUrl, emp);
  }

  update(id: number, emp: Employee): Observable<Employee> {
    return this.http.put<Employee>(`${this.apiUrl}/${id}`, emp);
  }

  delete(id: number): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/${id}`);
  }
}
```

Consuming in a Component

```
@Component({...})
export class EmployeeListComponent implements OnInit {
  employees: Employee[] = [];
```

```
loading = false;
error = '';

constructor(private empService: EmployeeService) {}

ngOnInit(): void {
  this.loading = true;
  this.empService.getAll().subscribe({
    next: data => { this.employees = data; this.loading = false; },
    error: err => { this.error = err.message; this.loading = false; }
  });
}
```

Headers, Params and Options

```
import { HttpHeaders, HttpParams } from '@angular/common/http';

const headers = new HttpHeaders()
  .set('Authorization', 'Bearer ' + token)
  .set('Content-Type', 'application/json');

const params = new HttpParams()
  .set('page', '0')
  .set('size', '10');

this.http.get<any>(this.apiUrl, { headers, params }).subscribe(...);
```


17. Observables and RxJS

RxJS (Reactive Extensions for JavaScript) provides Observables - a powerful way to handle asynchronous data streams. Angular uses RxJS heavily: HttpClient, Router, Reactive Forms, and EventEmitter all return Observables.

Observable vs Promise

Aspect	Promise	Observable
Values	Single value	Stream of values (0..n)
Lazy	Eager (runs immediately)	Lazy (runs on subscribe)
Cancellable	No	Yes (unsubscribe)
Operators	Few (then, catch)	Many (map, filter, mergeMap...)
Retry	Manual	Built-in retry, retryWhen

Creating and Subscribing

```
import { Observable, of, from, interval } from 'rxjs';

// Create
const obs$ = new Observable<number>(subscriber => {
  subscriber.next(1);
  subscriber.next(2);
  subscriber.complete();
});

// Shortcuts
of(1, 2, 3).subscribe(v => console.log(v));
from([10, 20, 30]).subscribe(v => console.log(v));
interval(1000).subscribe(n => console.log(n));
```

Common Operators

```
import { map, filter, tap, catchError, debounceTime,
  distinctUntilChanged, switchMap, mergeMap, take } from 'rxjs/operators';

this.http.get<Employee[]>(url).pipe(
  tap(data => console.log('Received', data)),
  map(list => list.filter(e => e.salary > 50000)),
  catchError(err => { console.error(err); return of([]); })
).subscribe(filtered => this.employees = filtered);
```

switchMap vs mergeMap vs concatMap

- switchMap:** cancels the previous inner Observable when a new one comes in. Best for search-as-you-type.
- mergeMap (flatMap):** runs all inner Observables concurrently. Best for independent parallel requests.
- concatMap:** waits for the previous to finish before starting the next. Best when order matters.
- exhaustMap:** ignores new emissions while the current inner is active. Best for login-button-spam scenarios.

Subject, BehaviorSubject, ReplaySubject

Subject: Multicast - same emission goes to all subscribers. No initial value.

BehaviorSubject: Stores current value; new subscribers immediately get it.

ReplaySubject: Replays last N values to new subscribers.

AsyncSubject: Only emits the last value when complete() is called.

```
import { BehaviorSubject } from 'rxjs';

@Injectable({ providedIn: 'root' })
export class AuthService {
  private currentUser = new BehaviorSubject<User | null>(null);
  currentUser$ = this.currentUser.asObservable();

  login(user: User) { this.currentUser.next(user); }
  logout() { this.currentUser.next(null); }
}
```

Unsubscribing - Avoid Memory Leaks

```
// Pattern 1: manual
private sub = new Subscription();
ngOnInit() {
  this.sub.add(this.service.data$.subscribe(...));
}
ngOnDestroy() { this.sub.unsubscribe(); }

// Pattern 2: takeUntil
private destroy$ = new Subject<void>();
ngOnInit() {
  this.service.data$.pipe(takeUntil(this.destroy$)).subscribe(...);
}
ngOnDestroy() { this.destroy$.next(); this.destroy$.complete(); }

// Pattern 3: async pipe (best in templates)
<!-- no manual unsubscribe needed -->
<li *ngFor="let e of employees$ | async">{{ e.name }}</li>
```

18. Interceptors and Error Handling

An HTTP Interceptor lets you inspect and transform every outgoing request and incoming response. Perfect for attaching auth tokens, logging, or centralized error handling.

Auth Interceptor

```
import { Injectable } from '@angular/core';
import { HttpInterceptor, HttpRequest, HttpHandler,
        HttpEvent } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler): Observable<HttpEvent<any>>&#x26;#x21D2;
    const token = localStorage.getItem('token');
    if (token) {
      const cloned = req.clone({
        headers: req.headers.set('Authorization', `Bearer ${token}`)
      });
      return next.handle(cloned);
    }
    return next.handle(req);
  }
}

// register in AppModule providers
providers: [
  { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true }
]
```

Error Interceptor

```
@Injectable()
export class ErrorInterceptor implements HttpInterceptor {
  constructor(private router: Router) {}

  intercept(req: HttpRequest<any>, next: HttpHandler) {
    return next.handle(req).pipe(
      catchError((err: HttpResponse) => {
        if (err.status === 401) this.router.navigate(['/login']);
        if (err.status === 500) console.error('Server error');
        return throwError(() => err);
      })
    );
  }
}
```

19. Authentication and JWT (with Spring Boot)

A typical flow: user submits credentials -> Angular calls /api/auth/login -> Spring Boot validates and returns a JWT -> Angular stores it (commonly in localStorage) and sends it in the Authorization header on each subsequent request.

Auth Service

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private apiUrl = 'http://localhost:8080/api/auth';
  private userSubject = new BehaviorSubject<User | null>(this.getUserFromStorage());
  user$ = this.userSubject.asObservable();

  constructor(private http: HttpClient, private router: Router) {}

  login(username: string, password: string) {
    return this.http.post<{token: string, user: User}>(
      `${this.apiUrl}/login`, { username, password }
    ).pipe(
      tap(res => {
        localStorage.setItem('token', res.token);
        localStorage.setItem('user', JSON.stringify(res.user));
        this.userSubject.next(res.user);
      })
    );
  }

  logout() {
    localStorage.removeItem('token');
    localStorage.removeItem('user');
    this.userSubject.next(null);
    this.router.navigate(['/login']);
  }

  isLoggedIn(): boolean { return !!localStorage.getItem('token'); }
  getToken(): string | null { return localStorage.getItem('token'); }

  private getUserFromStorage(): User | null {
    const u = localStorage.getItem('user');
    return u ? JSON.parse(u) : null;
  }
}
```

Login Component

```
@Component({...})
export class LoginComponent {
  form = this.fb.group({
    username: ['', Validators.required],
    password: ['', Validators.required]
  });
  error = '';

  constructor(private fb: FormBuilder,
    private auth: AuthService,
    private router: Router) {}

  onSubmit() {
```

```
    if (this.form.invalid) return;
    const { username, password } = this.form.value;
    this.auth.login(username!, password!).subscribe({
      next: () => this.router.navigate(['/dashboard']),
      error: err => this.error = 'Invalid credentials'
    });
  }
}
```

Role-Based Guard

```
@Injectable({ providedIn: 'root' })
export class RoleGuard implements CanActivate {
  constructor(private auth: AuthService, private router: Router) {}

  canActivate(route: ActivatedRouteSnapshot): boolean {
    const requiredRole = route.data['role'];
    const user = this.auth.currentUser;
    if (user && user.role === requiredRole) return true;
    this.router.navigate(['/forbidden']);
    return false;
  }
}

// usage
{ path: 'admin', component: AdminComponent,
  canActivate: [RoleGuard], data: { role: 'ADMIN' } }
```

20. Angular + Spring Boot Full Stack Integration

A typical Java full stack project has Angular running on the frontend (port 4200) and Spring Boot on the backend (port 8080). They communicate via REST APIs.

Backend (Spring Boot) - Quick Reference

```
// EmployeeController.java
@RestController
@RequestMapping("/api/employees")
@CrossOrigin(origins = "http://localhost:4200")
public class EmployeeController {
    @Autowired
    private EmployeeService service;

    @GetMapping
    public List<Employee> getAll() { return service.findAll(); }

    @GetMapping("/{id}")
    public Employee getById(@PathVariable Long id) {
        return service.findById(id);
    }

    @PostMapping
    public Employee create(@RequestBody @Valid Employee emp) {
        return service.save(emp);
    }

    @PutMapping("/{id}")
    public Employee update(@PathVariable Long id, @RequestBody Employee emp) {
        return service.update(id, emp);
    }

    @DeleteMapping("/{id}")
    public void delete(@PathVariable Long id) { service.delete(id); }
}
```

CORS Configuration

Browsers block cross-origin requests by default. The backend must allow the frontend origin. Options: `@CrossOrigin` annotation on controller, or a global `WebMvcConfigurer`.

```
// Global CORS config
@Configuration
public class CorsConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("http://localhost:4200")
            .allowedMethods("GET", "POST", "PUT", "DELETE")
            .allowedHeaders("*")
            .allowCredentials(true);
    }
}
```

Environment-Specific API URLs

```
// src/environments/environment.ts
export const environment = {
```

```
    production: false,
    apiUrl: 'http://localhost:8080/api'
  };

// src/environments/environment.prod.ts
export const environment = {
  production: true,
  apiUrl: 'https://api.mycompany.com/api'
};

// in service
import { environment } from '../environments/environment';
private apiUrl = environment.apiUrl + '/employees';
```

Common Full Stack Project Structure

```
my-fullstack-app/
  backend/                                <- Spring Boot
    src/main/java/com/company/
      controller/
      service/
      repository/
      model/
      config/
      security/
    pom.xml
  frontend/                               <- Angular
    src/app/
      core/      <- singleton services, guards, interceptors
      shared/    <- shared components, pipes, directives
      features/  <- feature modules (employees, orders, etc.)
      app.module.ts
      angular.json
```

Production Deployment Options

- **Separate deployment:** Angular on a CDN/Nginx, Spring Boot on a separate server. Most common.
- **Bundled deployment:** Build Angular into Spring Boot's static resources folder so one JAR serves both. Useful for small apps.
- **Docker:** Two containers (one for each) orchestrated with docker-compose or Kubernetes.

21. State Management Basics

For small apps, services with BehaviorSubject are enough. For larger apps with complex shared state, consider a state management library like NgRx.

Service-Based State (Recommended for L1/L2)

```
@Injectable({ providedIn: 'root' })
export class CartService {
  private itemsSubject = new BehaviorSubject<CartItem[]>([]);
  items$ = this.itemsSubject.asObservable();

  get items(): CartItem[] { return this.itemsSubject.value; }

  add(item: CartItem) {
    this.itemsSubject.next([...this.items, item]);
  }

  remove(id: number) {
    this.itemsSubject.next(this.items.filter(i => i.id !== id));
  }

  total$ = this.items$.pipe(
    map(items => items.reduce((s, i) => s + i.price * i.qty, 0))
  );
}
```

NgRx (Overview Only)

- Redux-inspired state management for Angular.
- Core pieces: Store (single source of truth), Actions (events), Reducers (pure state transitions), Selectors (derived data), Effects (side effects like HTTP calls).
- Best for large applications with complex shared state.
- For L1/L2 interviews you mainly need awareness; deep NgRx is L3+ material.

22. Unit Testing Essentials

Angular ships with Jasmine (testing framework) and Karma (test runner). Each generated component or service comes with a .spec.ts file.

Testing a Component

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { HelloComponent } from './hello.component';

describe('HelloComponent', () => {
  let component: HelloComponent;
  let fixture: ComponentFixture<HelloComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      declarations: [HelloComponent]
    }).compileComponents();
    fixture = TestBed.createComponent(HelloComponent);
    component = fixture.componentInstance;
    fixture.detectChanges();
  });

  it('should create', () => {
    expect(component).toBeTruthy();
  });

  it('should display the title', () => {
    component.title = 'Hi';
    fixture.detectChanges();
    const el = fixture.nativeElement.querySelector('h1');
    expect(el.textContent).toContain('Hi');
  });
});
```

Testing a Service with HttpClient

```
import { HttpClientTestingModule, HttpTestingController } from
  '@angular/common/http/testing';

describe('EmployeeService', () => {
  let service: EmployeeService;
  let httpMock: HttpTestingController;

  beforeEach(() => {
    TestBed.configureTestingModule({
      imports: [HttpClientTestingModule],
      providers: [EmployeeService]
    });
    service = TestBed.inject(EmployeeService);
    httpMock = TestBed.inject(HttpTestingController);
  });

  it('should fetch employees', () => {
    const mock = [{ id: 1, name: 'A' }];
    service.getAll().subscribe(res => expect(res).toEqual(mock));
    const req = httpMock.expectOne(service['apiUrl']);
    expect(req.request.method).toBe('GET');
    req.flush(mock);
  });
});
```

```
    });  
  });
```

23. Build, Optimization and Deployment

Building for Production

```
ng build --configuration production
# outputs to dist/<project-name>/
```

Key Optimizations Angular Does for You

- **AOT (Ahead-of-Time) compilation:** Templates compiled at build time -> smaller, faster.
- **Tree shaking:** Unused code is removed.
- **Minification:** JavaScript and CSS are minified.
- **Bundling:** Code is split into chunks (main, vendor, lazy chunks).
- **Differential loading:** Modern and legacy bundles for different browsers.

Performance Best Practices

- Use lazy loading for feature modules.
- Use OnPush change detection where appropriate.
- Use trackBy in *ngFor to avoid rebuilding the entire list.
- Unsubscribe from Observables (or use async pipe).
- Avoid heavy computations in templates - use pipes or memoization.
- Use pure pipes; reach for impure ones only when truly necessary.
- Compress images and use modern formats (WebP).

Deployment

- Copy contents of dist/ to your web server (Nginx, Apache, S3 + CloudFront).
- Configure the server to serve index.html for all unknown routes (SPA fallback).
- Set proper cache headers: long for hashed files, no-cache for index.html.
- Enable Gzip / Brotli compression.

24. Best Practices and Angular Style Guide

- Follow the official Angular Style Guide naming conventions: kebab-case filenames, PascalCase class names, .component.ts / .service.ts suffixes.
- Keep components small and focused (Single Responsibility).
- Move business logic out of components into services.
- Use strong types - avoid 'any' wherever possible.
- Use Reactive Forms for non-trivial forms.
- Use environment files for configuration.
- Organize code by feature, not by type (folders like users/, orders/, not all-services/).
- Use a CoreModule for app-wide singletons, SharedModule for common UI building blocks.
- Lazy load feature modules.
- Always unsubscribe from manual subscriptions.
- Write meaningful unit tests for services and key components.
- Use Angular Linter (ng lint) and Prettier for consistent formatting.
- Never commit sensitive data (API keys, tokens) in the repository.

25. L1 Interview Questions (Beginner)

These cover fundamentals interviewers expect every Angular developer to know. Answers are concise and crisp - perfect for verbal responses.

Q: What is Angular?

A: Angular is a TypeScript-based, open-source front-end framework developed by Google for building single-page applications. It provides a component-based architecture, two-way data binding, dependency injection, routing, forms, and HTTP communication out of the box.

Q: What is the difference between AngularJS and Angular?

A: AngularJS (v1.x) uses JavaScript and MVC architecture. Angular (v2+) is a complete rewrite using TypeScript with a component-based architecture, better performance, mobile support, and a powerful CLI.

Q: What is TypeScript and why does Angular use it?

A: TypeScript is a superset of JavaScript that adds static typing, interfaces, decorators, and modern features. Angular uses it for type safety, better tooling (autocomplete, refactoring), and to catch errors at compile time.

Q: What is a component?

A: A component is the basic building block of an Angular UI. It consists of a TypeScript class (logic), an HTML template (view), and CSS styles. Components are marked with the `@Component` decorator.

Q: What is a module (NgModule)?

A: An NgModule groups related components, directives, pipes, and services. Every Angular app has at least one root module - AppModule. Modules help organize code and enable features like lazy loading.

Q: What is data binding? Name the types.

A: Data binding synchronizes data between the component class and the view. Types: Interpolation `{{ }}`, Property binding `[prop]`, Event binding (event), Two-way binding `[(ngModel)]`.

Q: What is the difference between property binding and interpolation?

A: Interpolation `{{ value }}` is for inserting text into the HTML; it always produces a string. Property binding `[property]="value"` binds to an element's DOM property and can pass non-string types like booleans, objects, and functions.

Q: What are directives? Name the three kinds.

A: Directives add behavior to DOM elements. Three kinds: Components (directives with templates), Structural directives (`*ngIf`, `*ngFor` - change layout), Attribute directives (`ngClass`, `ngStyle` - change appearance/behavior).

Q: What is `*ngIf` and `*ngFor`?

A: `*ngIf` conditionally adds/removes an element from the DOM. `*ngFor` iterates over a collection and renders an element for each item.

Q: What are pipes?

A: Pipes transform displayed values in templates without modifying the underlying data. Examples: date, currency, uppercase, async. Custom pipes implement the PipeTransform interface.

Q: What is the async pipe?

A: The async pipe subscribes to an Observable or Promise, returns the latest emitted value, and automatically unsubscribes when the component is destroyed. It prevents memory leaks and reduces boilerplate.

Q: What is a service?

A: A service is a class that encapsulates reusable logic, data, or functionality (HTTP calls, business rules) that can be injected into components and other services.

Q: What is dependency injection in Angular?

A: DI is a design pattern where dependencies (services) are provided to a class rather than created by it. Angular's DI system creates and provides instances based on metadata, making code modular and testable.

Q: What is @Injectable() used for?

A: @Injectable() marks a class as available to be provided and injected as a dependency. The providedIn: 'root' option registers it as an app-wide singleton.

Q: What are Angular lifecycle hooks?

A: Methods Angular calls at specific points in a component's life: ngOnInit (after first inputs set), ngOnChanges (inputs changed), ngOnDestroy (before destruction), ngAfterViewInit (view ready), etc.

Q: What is the purpose of ngOnInit?

A: ngOnInit runs once after Angular has initialized the component's inputs. It's the standard place for initialization logic like fetching data or setting up subscriptions.

Q: What is the difference between constructor and ngOnInit?

A: Constructor is a TypeScript feature that runs when the class is instantiated; use it only for simple field initialization and dependency injection. ngOnInit is an Angular lifecycle hook that runs after Angular has set the component's inputs - use it for initialization logic.

Q: What is an Observable?

A: An Observable is a stream of asynchronous data over time, provided by RxJS. Unlike Promises (single value, eager), Observables can emit multiple values, are lazy, and can be cancelled.

Q: Promise vs Observable - main differences?

A: Promise: single value, eager, not cancellable. Observable: multiple values over time, lazy (runs on subscribe), cancellable via unsubscribe, supports rich operators (map, filter, switchMap).

Q: What is the difference between Reactive and Template-driven forms?

A: Template-driven forms define the form model in the template using ngModel directives - simpler, good for basic forms. Reactive forms define the model in the TypeScript class using FormGroup/FormControl - more powerful, testable, suited for complex forms.

Q: What is routing in Angular?

A: Routing enables navigation between views without full page reloads. The Angular Router maps URL paths to components, supporting parameters, child routes, lazy loading, and guards.

Q: What is a route guard?

A: A guard is a service that controls whether a user can access a route. Types: CanActivate, CanActivateChild, CanDeactivate, CanLoad, Resolve. Common use: blocking unauthenticated users from protected routes.

Q: What is HttpClient used for?

A: HttpClient (from @angular/common/http) is Angular's API for making HTTP requests to REST endpoints. It returns Observables and supports typed responses, headers, interceptors, and error handling.

Q: What is an HTTP Interceptor?

A: An interceptor sits between your app and the backend, transforming outgoing requests and incoming responses. Common uses: adding auth tokens, logging, centralized error handling.

Q: What is @Input and @Output?

A: @Input lets a parent component pass data to a child via property binding. @Output lets a child emit events to the parent via an EventEmitter.

Q: What is the selector property in @Component?

A: selector defines the CSS selector used in HTML to insert the component. For example, selector: 'app-employee' means you use in templates.

Q: What is ng-content?

A: ng-content enables content projection - it acts as a placeholder where a parent component can insert custom HTML inside a child component's template, like passing children in other frameworks.

Q: What is the difference between ngOnChanges and ngOnInit?

A: ngOnChanges runs every time an @Input property changes (called with a SimpleChanges object). ngOnInit runs once after the first ngOnChanges - so by the time ngOnInit runs, inputs are already set.

Q: What is a SPA (Single Page Application)?

A: A SPA is a web app that loads a single HTML page and dynamically updates the content as the user interacts with it, without full page reloads. Angular is designed for SPAs.

Q: What is bootstrapping in Angular?

A: Bootstrapping is the process of loading the root module (AppModule) and starting the app by rendering the root component. This happens in main.ts via platformBrowserDynamic().bootstrapModule(AppModule).

26. L2 Interview Questions (Intermediate)

These dig deeper into how Angular works internally and how to apply it in real projects. Expect follow-ups about trade-offs and alternatives.

Q: Explain change detection in Angular.

A: Change detection is the mechanism that keeps the view in sync with the component state. Angular's default strategy checks every component in the tree on each async event (clicks, HTTP, timers) triggered by Zone.js. You can opt into OnPush strategy to only check a component when its inputs change by reference, an event fires inside it, or an observable bound via async pipe emits.

Q: What is OnPush change detection and when would you use it?

A: OnPush tells Angular to skip change detection unless the component's inputs change by reference, an event triggers inside it, or an async pipe emits. It dramatically improves performance for large component trees. Use it with immutable data patterns.

Q: Explain hierarchical Dependency Injection.

A: Angular maintains a tree of injectors that mirrors the component tree. When a class requests a dependency, Angular looks up the tree starting from the local injector and walks upward to the root. This allows scoping services to specific component subtrees and overriding services for specific branches.

Q: Difference between providedIn: 'root' vs declaring in module providers vs component providers?

A: providedIn: 'root' creates a single, tree-shakeable, app-wide instance. Listing a service in a module's providers also creates a single instance (for eagerly loaded modules) but is not tree-shakeable. Component providers create a new instance per component instance and its subtree - useful when each instance needs its own state.

Q: What is the difference between Subject, BehaviorSubject, and ReplaySubject?

A: Subject is a multicast Observable - emits to all current subscribers but new subscribers miss past values. BehaviorSubject stores a current value and immediately emits it to new subscribers (needs initial value). ReplaySubject replays the last N values to new subscribers. AsyncSubject only emits the last value when complete() is called.

Q: Explain switchMap, mergeMap, concatMap, and exhaustMap.

A: All flatten an inner Observable into the outer stream. switchMap cancels the previous inner Observable when a new one starts (great for type-ahead search). mergeMap runs all inner Observables concurrently (great for independent parallel work). concatMap queues them and runs one at a time, preserving order. exhaustMap ignores new emissions while one is in flight (good for login button spam).

Q: What is lazy loading and how do you implement it?

A: Lazy loading defers loading a feature module's code until the user navigates to its route. Implement it by using loadChildren in the route config with a dynamic import: { path: 'admin', loadChildren: () => import('./admin/admin.module').then(m => m.AdminModule) }. The module gets its own bundle and loads on demand.

Q: What is AOT compilation and how does it differ from JIT?

A: AOT (Ahead-of-Time) compiles Angular HTML templates and TypeScript at build time, producing optimized JavaScript. JIT (Just-in-Time) compiles in the browser at runtime. AOT gives faster rendering, smaller bundles, earlier template error detection, and better security. Production builds use AOT by default.

Q: How do you prevent memory leaks from Observables?

A: Always unsubscribe from manually subscribed Observables. Common patterns: store a Subscription and call `.unsubscribe()` in `ngOnDestroy`; use the `takeUntil(destroy$)` pattern; or prefer the async pipe in templates, which auto-unsubscribes.

Q: Explain the role of NgZone and Zone.js.

A: Zone.js monkey-patches async APIs (`setTimeout`, Promises, events, XHR) to notify Angular when async tasks finish. NgZone wraps this so Angular can trigger change detection at the right moments. You can use `NgZone.runOutsideAngular()` to run code without triggering change detection - useful for high-frequency operations like animations.

Q: How does Angular handle forms validation? Compare built-in vs custom validators.

A: Built-in validators come from `Validators` (`required`, `minLength`, `maxLength`, `email`, `pattern`). They are functions that return null if valid or a `ValidationErrors` object if not. Custom validators follow the same contract; they can be synchronous (`AbstractControl` -> `ValidationErrors` | null) or asynchronous (returns `Promise` / `Observable`). Validators are attached when creating `FormControls`.

Q: What is the difference between FormGroup, FormControl, and FormArray?

A: `FormControl` tracks the value and validity of a single input. `FormGroup` is a collection of `FormControls` (typically a form's structure). `FormArray` is an ordered, dynamic list of `FormControls/Groups` - useful when the number of fields is variable (e.g. adding rows).

Q: Explain content projection and ng-content.

A: Content projection lets you pass custom HTML from a parent into a child component's template using `<ng-content>`. With multi-slot projection, you can target specific slots using `ng-content` with a `select` attribute (e.g. `select=".header"`). This makes components flexible and reusable.

Q: What is the difference between ViewChild and ContentChild?

A: `@ViewChild` queries an element/component from the component's own template. `@ContentChild` queries content projected into the component via `ng-content`. `ViewChild` is available in `ngAfterViewInit`; `ContentChild` in `ngAfterContentInit`.

Q: Explain Angular's bootstrapping process.

A: `main.ts` calls `platformBrowserDynamic().bootstrapModule(AppModule)`. Angular initializes the root `NgModule`, resolves its providers, and bootstraps the components listed in the module's `bootstrap` array (typically `AppComponent`). The selector of `AppComponent` in `index.html` is replaced with the rendered component.

Q: How do interceptors work and can multiple interceptors be chained?

A: Interceptors implement `HttpInterceptor` with an `intercept(req, next)` method. They can modify the request or response, then call `next.handle(req)` to pass it on. Yes, multiple interceptors can be registered (each with `multi: true`); Angular runs them in the order they're provided for requests, and in reverse order for responses.

Q: What are Resolvers and when do you use them?

A: A Resolver implements the Resolve interface and fetches data before a route is activated, so the component receives the data ready in the route's data. Use them when you don't want to show a partial UI while waiting on data.

Q: Explain trackBy in *ngFor and why it matters.

A: By default, *ngFor uses object identity to detect changes; if the list reference changes, Angular re-renders everything. trackBy lets you provide a function that returns a unique key (typically id), so Angular only re-renders rows whose key changed. Important for performance in large lists.

Q: What is the difference between pure and impure pipes?

A: Pure pipes (default) are re-evaluated only when the input reference changes - efficient. Impure pipes run on every change detection cycle - flexible but can hurt performance. AsyncPipe is impure under the hood.

Q: How do you share data between sibling components?

A: Use a shared service that holds a Subject or BehaviorSubject. The sender calls .next() and the receiver subscribes to the exposed Observable. This avoids tight coupling and works across the component tree.

Q: Explain Angular's security mechanisms (XSS, sanitization).

A: Angular treats values bound to the DOM as untrusted by default and sanitizes them, stripping potentially dangerous content. DomSanitizer.bypassSecurityTrust* methods can be used to bypass sanitization when you genuinely trust the source. Also: avoid innerHTML where possible; never concatenate untrusted strings into URLs or HTML.

Q: How would you optimize a slow Angular application?

A: Profile first with Angular DevTools. Then apply: OnPush change detection, trackBy in *ngFor, lazy loading feature modules, async pipe over manual subscriptions, immutable data, virtual scrolling for large lists (cdk-virtual-scroll), pure pipes for derived values, removing console.logs and unused libraries, and a production build with AOT.

Q: How does Angular communicate with a Spring Boot backend?

A: Angular's HttpClient calls REST endpoints exposed by Spring Boot controllers (e.g. /api/employees). Backend must allow CORS for the Angular origin. Auth typically uses JWT: backend returns a token on login; Angular stores it and attaches it on every request via an interceptor. Use environment files to switch URLs between dev and prod.

Q: What is the role of environment.ts files?

A: They hold environment-specific configuration (API URLs, feature flags). At build time, Angular replaces environment.ts with environment.prod.ts (or another configuration) so code reads the right values without hardcoding.

Q: How do you implement role-based access control?

A: Decode the JWT on the client to know the user's role (treat it as a hint, not security). Use a CanActivate guard that compares the user's role against the role required by the route (stored in route.data). Hide UI elements with *ngIf based on role. Real enforcement must always happen on the server.

Q: What is the difference between snapshot and observable route params?

A: `route.snapshot paramMap.get('id')` reads the value once at activation time. `route.paramMap` is an Observable that emits each time the params change - important when the same component instance is reused for a different param (Angular reuses the component for navigations within the same route).

Q: What's the difference between hash-based and HTML5 routing?

A: HTML5 routing (default) produces clean URLs like `/users/1` using the History API; the server must serve `index.html` for unknown routes. Hash routing uses URLs like `/#/users/1`; it works on any static host without server config but is uglier and less SEO-friendly.

Q: How would you handle global error handling?

A: Two layers: an HTTP interceptor that catches API errors uniformly (logging, toast, 401 redirects); and a custom ErrorHandler service (providing { provide: ErrorHandler, useClass: GlobalErrorHandler }) to catch any uncaught JavaScript error in the app.

Q: What is a Standalone Component (Angular 14+)?

A: A standalone component declares its own imports and does not need to be part of an NgModule. Mark with `standalone: true` and list its imports in the `@Component` decorator. Modern Angular projects can be entirely module-less.

Q: Difference between ng build and ng serve?

A: `ng serve` runs a local dev server with live reload, builds in memory, and is meant for development. `ng build` produces compiled output in the `dist/` folder for deployment. `ng build --configuration production` applies AOT, minification, and other production optimizations.

27. Coding and Scenario-Based Questions

Interviewers often ask you to write small snippets or design solutions. Practice the following patterns until they feel automatic.

Q1. Implement debounced search-as-you-type.

```
// component.ts
searchControl = new FormControl('');
results$ = this.searchControl.valueChanges.pipe(
  debounceTime(300),
  distinctUntilChanged(),
  switchMap(term => this.api.search(term || ''))
);

<!-- template -->
<input [formControl]="searchControl">
<li *ngFor="let r of results$ | async">{{ r.name }}</li>
```

Q2. Show a loading spinner while an HTTP call is in progress.

```
loading = false;
ngOnInit() {
  this.loading = true;
  this.api.getData().pipe(finally(() => this.loading = false))
    .subscribe(d => this.data = d);
}

<!-- template -->
<div *ngIf="loading">Loading...</div>
```

Q3. Implement a confirm-leave guard.

```
export interface CanLeave { canLeave: () => boolean; }

@Injectable({ providedIn: 'root' })
export class LeaveGuard implements CanDeactivate<CanLeave> {
  canDeactivate(comp: CanLeave): boolean {
    return comp.canLeave() || confirm('You have unsaved changes. Leave?');
  }
}
```

Q4. Filter a list using a search term (pipe vs method).

Prefer a method or a getter (or computing in the component) over a custom impure pipe. Impure pipes run on every change detection cycle and hurt performance.

```
// component
search = '';
items = [...];
get filtered() {
  return this.items.filter(i =>
    i.name.toLowerCase().includes(this.search.toLowerCase()));
}

<!-- template -->
<input [(ngModel)]="search">
<li *ngFor="let i of filtered; trackBy: trackById">{{ i.name }}</li>
```

Q5. Forward a token in every HTTP request.

Use an `HttpInterceptor` that reads the token and clones the request with an `Authorization` header (see Section 18).

Q6. Pagination - call API with page and size.

```
loadPage(page: number) {  
  const params = new HttpParams()  
    .set('page', page).set('size', this.size);  
  this.api.get<Page<Item>>>(url, { params })  
    .subscribe(p => { this.items = p.content; this.total = p.totalElements; });  
}
```

Q7. Dynamic form with FormArray.

```
form = this.fb.group({  
  name: [''],  
  phones: this.fb.array([this.fb.control('')])  
});  
get phones() { return this.form.get('phones') as FormArray; }  
addPhone() { this.phones.push(this.fb.control('')); }  
removePhone(i: number) { this.phones.removeAt(i); }  
  
<!-- template -->  
<div formArrayName="phones">  
  <div *ngFor="let p of phones.controls; let i = index">  
    <input [formControlName]="i">  
    <button (click)="removePhone(i)">x</button>  
  </div>  
</div>  
<button (click)="addPhone()">Add phone</button>
```

Q8. Communicate from a deeply nested child to an ancestor.

Use a shared service with a Subject (preferred) rather than chaining @Output through every level. The child injects the service and calls .next(); the ancestor subscribes.

Q9. Show different views based on user role.

```
<div *ngIf="(auth.user$ | async)?.role === 'ADMIN'">  
  <admin-panel></admin-panel>  
</div>  
<div *ngIf="(auth.user$ | async)?.role === 'USER'">  
  <user-dashboard></user-dashboard>  
</div>
```

Q10. Avoid 'ExpressionChangedAfterItHasBeenCheckedError'.

This error occurs when a value changes during the same change detection cycle as it is read. Fixes: move the change to ngAfterViewInit and wrap in setTimeout or Promise.resolve(); use a BehaviorSubject and async pipe; or use detectChanges() carefully via ChangeDetectorRef.

28. Final Preparation Checklist

Concepts to Master Before the Interview

- Components, templates, data binding, directives, pipes.
- Services and Dependency Injection with hierarchical injectors.
- Lifecycle hooks (especially `ngOnInit` and `ngOnDestroy`).
- Component communication: `@Input`, `@Output`, `@ViewChild`, services.
- Modules and lazy loading.
- Routing, route params, guards, resolvers.
- Forms (both Reactive and Template-driven).
- `HttpClient` and interceptors.
- Observables, common RxJS operators, async pipe, memory leak patterns.
- Change detection basics (default vs `OnPush`).
- Authentication flow with JWT and Spring Boot.
- `AsyncPipe`, lazy loading, `trackBy`, async pipe for performance.
- Environment files and CORS for full stack apps.

Practical Things to Prepare

- Build a small CRUD app (e.g. Employee Manager) using Angular + Spring Boot. Be ready to talk through it.
- Be able to draw the project structure on a whiteboard.
- Practice writing a service with `HttpClient` from memory.
- Practice writing a Reactive Form with custom validation from memory.
- Practice writing an HTTP interceptor from memory.
- Know the difference between common operators (`map`, `switchMap`, `mergeMap`).
- Have a one-minute story for: 'Tell me about an Angular project you've worked on.'

Day-Before Quick Revision

- Re-read Section 25 (L1) and Section 26 (L2) once aloud.
- Skim the code snippets in Sections 6-19.
- Open one of your own projects and skim its structure.
- Sleep well. Confidence > cramming.

Best of luck with your interview! Be calm, be honest, and explain your thought process out loud. Interviewers value clear reasoning over perfect answers.